PHASE 9 — CRYPTOGRAPHIC KNOWLEDGE GRAPH & ENTERPRISE RELATIONSHIP MODEL

Project: Enterprise Cryptographic Discovery & Analysis Tool (ECDAT)

Problem Statement: SIH26164

Organization: National Technical Research Organisation (NTRO)

Theme: Blockchain & Cybersecurity

Phase: 9 of 22

Document Type: Technical Research & Engineering Handbook

Status: Enterprise Cryptographic Intelligence Graph

Table of Contents

22. Data Assets
23. Business Processes
24. Owners
25. Findings
26. Risks
27. Recommendations
28. Migration Plans
29. Node Metadata
30. Relationship Metadata
31. Evidence Graph
32. Dependency Graph
33. Cryptographic Usage Graph
34. PKI Trust Graph
35. Certificate Graph
36. Key Dependency Graph
37. Application-to-Crypto Graph
38. Data-to-Crypto Graph
39. Business-to-Crypto Graph
40. Risk Propagation
41. Blast Radius
42. Centrality
43. Single Points of Failure
44. Shared Cryptographic Dependencies
45. Attack Path Analysis
46. Migration Dependency Graph
47. Crypto-Agility Graph
48. Knowledge Graph Queries
49. Natural Language Queries
50. Example Query 1
51. Example Query 2
52. Example Query 3
53. Example Query 4
54. Example Query 5

55. Graph + AI
56. Graph RAG
57. AI Graph Reasoning
58. Graph-Based Recommendations
59. Graph-Based Risk
60. Temporal Graph
61. Historical Tracking
62. Change Detection
63. Versioning
64. Graph Data Quality
65. Deduplication
66. Entity Resolution
67. Confidence
68. Provenance
69. Graph Database Architecture
70. Neo4j / Property Graph
71. Relational Database Integration
72. Vector Database Integration
73. Search Architecture
74. Graph API
75. Graph Visualization
76. Enterprise Dashboard
77. Security
78. Access Control
79. Sensitive Relationships
80. Graph Threat Model
81. Scalability
82. Graph Processing Pipeline
83. Complete Architecture
84. Requirements Derived From Phase 9
85. Phase 9 Summary
86. Phase 10 Preview

---

1. Purpose of This Phase

The previous phases built the core intelligence pipeline:

```
Cryptography
    ↓
Quantum Threat
    ↓
PQC
    ↓
Enterprise Ecosystem
    ↓
Discovery
    ↓
CBOM
    ↓
Risk Assessment
    ↓
PQC Recommendation
    ↓
AI Code Analysis
```

At this point ECDAT can discover individual cryptographic assets.

But an enterprise rarely has isolated assets.

Instead:

```
Application
   ↓
Library
   ↓
Crypto Algorithm
   ↓
Certificate
   ↓
CA
   ↓
HSM
```

and:

```
Application
   ↓
API
   ↓
Database
   ↓
Sensitive Data
```

and:

```
Application
   ↓
Business Process
   ↓
Business Unit
   ↓
Owner
```

Everything is connected.

This phase designs the system that understands those relationships.

The Cryptographic Knowledge Graph.

---

2. Why a Knowledge Graph?

A traditional inventory might say:

```
RSA-2048
Used in:
Application A
Application B
Application C
```

A graph can say:

```
RSA-2048
│
├── used by → Application A
│              └── serves → Payment Processing
│
├── used by → Application B
│              └── protects → Customer Data
```

```
    │
    └── used by → Application C
                  └── certificate signed by → CA-01
```

The second representation contains **relationships**, not just records.

This allows ECDAT to answer questions that an ordinary inventory struggles with.

---

3. From Inventory to Relationships

The fundamental evolution is:

```
Inventory
    ↓
Relationships
    ↓
Context
    ↓
Impact
    ↓
Intelligence
```

For example:

```
"RSA-2048 exists"
```

is useful.

But:

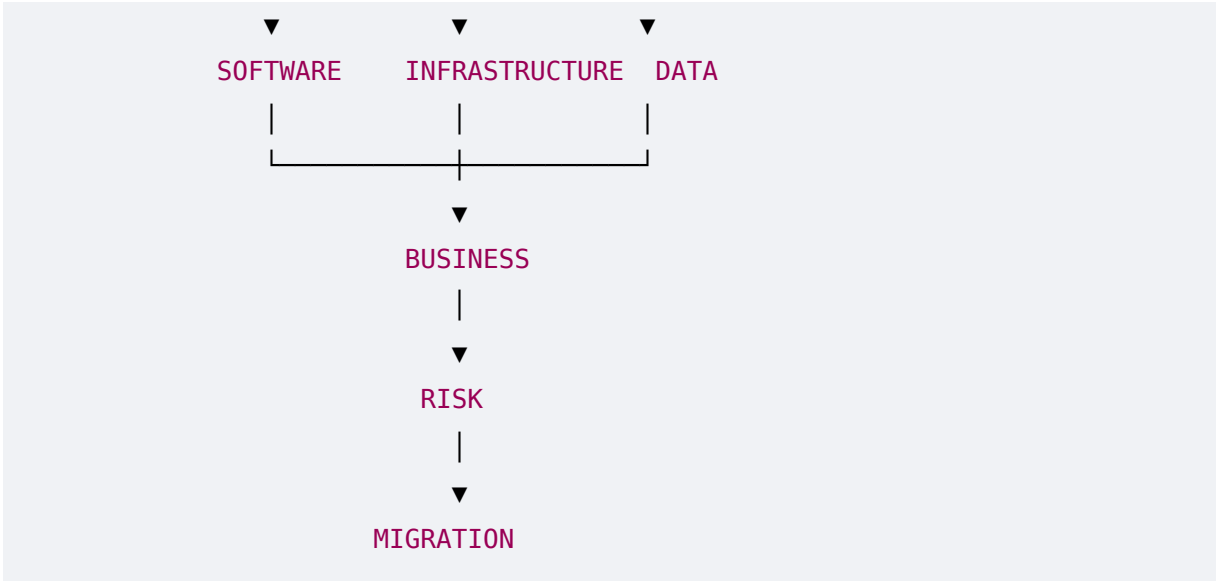
```
"RSA-2048 is used by 73 critical applications,
including the payment gateway and certificate
infrastructure, and migration is blocked by
a shared HSM dependency."
```

is much more valuable.

---

4. The ECDAT Intelligence Model

The graph should connect five major domains:



This allows technical discoveries to be connected to business consequences.

---

5. Graph Fundamentals

ECDAT can use a **property graph** model.

The basic components are:

```
Node
Relationship
Property
Evidence
```

Example:

```
(Application)
      |
      | USES
      ▼
(CryptographicAlgorithm)
```

Each node and relationship can contain metadata.

---

6. Nodes

A node represents an entity.

Examples:

```
Application
Repository
```

```
SourceFile
Function
Library
Algorithm
Key
Certificate
CA
Protocol
Container
Server
HSM
CloudService
DataAsset
BusinessProcess
Owner
```

---

7. Relationships

Relationships explain how entities interact.

Examples:

```
USES
DEPENDS_ON
CONTAINS
PROTECTS
SIGNED_BY
ISSUED_BY
STORED_IN
RUNS_ON
CALLS
IMPLEMENTS
DEPLOYED_TO
OWNED_BY
AFFECTS
REQUIRES
RECOMMENDS
MIGRATES_TO
```

---

8. Applications

Application nodes should contain:

```
Application ID
Name
Version
Environment
Criticality
Owner
Business Unit
Technology
Deployment
```

Example:

```
Application:
Payment Gateway

Criticality:
Mission Critical

Environment:
Production
```

---

9. Repositories

A repository represents a source-code project.

Properties:

```
Repository URL
Branch
Commit
Language
Owner
Last Scan
Visibility
```

Relationship:

```
Repository
    |
    └── CONTAINS
          ↓
       SourceFile
```

---

10. Source Files

A source file can contain:

```
Path
Language
Hash
Repository
Commit
```

Example:

```
src/security/encryption.py
```

Relationships:

```
SourceFile
   ↓
DEFINES
   ↓
Function
```

---

11. Functions

Functions are important because cryptographic operations frequently occur at function level.

Example:

```
encrypt_customer_data()
```

Properties:

```
Name
File
Line
Language
Signature
```

Relationships:

```
Function
   ↓
CALLS
```

```
   ↓
CryptoFunction
```

---

12. Libraries

Library nodes represent dependencies.

Example:

```
OpenSSL
Bouncy Castle
libsodium
PyCA Cryptography
```

Properties:

```
Name
Version
Package Manager
License
Source
```

Relationships:

```
Application
   ↓
DEPENDS_ON
   ↓
Library
```

---

13. Cryptographic Algorithms

This is one of the central node types.

Examples:

```
RSA
ECDSA
ECDH
AES
ChaCha20
SHA-256
ML-KEM
```

```
ML-DSA
SLH-DSA
```

Properties:

```
Family
Function
Key Size
Security Status
Quantum Status
Standard
Lifecycle
```

---

14. Keys

Key nodes can represent metadata about keys without exposing the secret material.

Example:

```
Key ID
Key Type
Algorithm
Size
Purpose
Location
Rotation Policy
Owner
```

Important:

> ECDAT should generally inventory key metadata, not extract private key material.

---

15. Certificates

Certificate nodes can contain:

```
Subject
Issuer
Serial
Algorithm
Public Key Type
Validity
```

```
SANs
Key Usage
Environment
```

Relationships:

```
Certificate
    │
    ├── USES → Algorithm
    ├── ISSUED_BY → CA
    └── PROTECTS → Application
```

---

16. Certificate Authorities

CA nodes represent trust infrastructure.

Example:

```
Root CA
Intermediate CA
Issuing CA
```

Relationships:

```
Root CA
   ↓
ISSUES
   ↓
Intermediate CA
   ↓
ISSUES
   ↓
Certificate
```

This creates a PKI trust graph.

---

17. Protocols

Protocol nodes include:

```
TLS
IPsec
SSH
```

```
S/MIME
Kerberos
Application-specific protocols
```

Properties:

```
Version
Configuration
Cipher Suites
Key Exchange
Authentication
```

---

18. Containers

Container nodes:

```
Image
Tag
Digest
Base Image
Packages
Libraries
```

Relationships:

```
Container
   ↓
CONTAINS
   ↓
Library
```

This allows ECDAT to discover crypto embedded in container images.

---

19. Infrastructure

Infrastructure nodes include:

```
Server
VM
Cluster
Network Appliance
Load Balancer
```

```
Firewall
Gateway
```

Relationships:

```
Application
   ↓
DEPLOYED_ON
   ↓
Infrastructure
```

---

20. Cloud Services

Cloud nodes could represent:

```
KMS
HSM
Certificate Service
Load Balancer
API Gateway
Managed Database
Container Service
```

Example:

```
Application
    ↓
USES
    ↓
Cloud KMS
```

---

21. HSMs

HSMs deserve special treatment.

An HSM may support:

```
Key Storage
Signing
Encryption
Certificate Operations
Key Generation
```

Graph:

```
HSM
│
├── STORES → Key
├── USED_BY → Application
└── SUPPORTS → Algorithm
```

---

22. Data Assets

Data nodes represent information being protected.

Examples:

```
Customer Data
Government Records
Financial Data
Credentials
Intellectual Property
Strategic Information
```

Properties:

```
Sensitivity
Retention
Classification
Owner
Location
```

---

23. Business Processes

This is where technical security connects to business.

Example:

```
Business Process:
Online Payment Processing
```

Relationships:

```
Business Process
   ↓
DEPENDS_ON
```

```
      ↓
Payment Application
```

Then:

```
Payment Application
   ↓
USES
   ↓
ECDSA
```

Now cryptographic risk can propagate into business context.

---

24. Owners

Owner nodes can represent:

```
Team
Department
Application Owner
Security Owner
Infrastructure Owner
Vendor
```

Example:

```
Application
   ↓
OWNED_BY
   ↓
Payments Team
```

This enables accountability.

---

25. Findings

A finding represents an observation.

Example:

```
Finding:
ECDSA detected in signing service.
```

Properties:

```
Severity
Confidence
Evidence
Scanner
Timestamp
Status
```

---

26. Risks

Risk nodes represent assessed risk.

Example:

```
Risk:
Critical Quantum Exposure
```

Relationships:

```
Risk
 ↓
AFFECTS
 ↓
Application
```

---

27. Recommendations

Recommendation nodes capture decisions.

Example:

```
Recommendation:
Migrate ECDSA → ML-DSA
```

Properties:

```
Algorithm
Reason
Confidence
Constraints
Status
Created
```

---

28. Migration Plans

A migration plan describes:

```
Current State
Target State
Dependencies
Sequence
Timeline
Owner
Status
```

Graph:

```
Legacy Algorithm
      ↓
MIGRATES_TO
      ↓
PQC Algorithm
```

---

29. Node Metadata

Every important node should contain:

```
id
type
name
source
confidence
created_at
updated_at
environment
classification
```

Example:

```
{
  "id": "crypto-00123",
  "type": "algorithm",
  "name": "ECDSA",
  "confidence": 0.98,
```

```
  "source": "static-analysis"
}
```

---

30. Relationship Metadata

Relationships also need metadata.

Example:

```
Application
   |
   | USES
   |
ECDSA
```

Relationship properties:

```
Evidence
Confidence
Source
First Seen
Last Seen
Environment
```

This matters because relationships can change.

---

31. Evidence Graph

Every important graph relationship should be backed by evidence.

Example:

```
Application A
   |
   | USES
   |
ECDSA
```

Evidence:

```
src/signing/service.py:42
```

Another relationship:

```
Application A
   |
   | DEPENDS_ON
   |
OpenSSL
```

Evidence:

```
container manifest
```

This makes the graph auditable.

---

32. Dependency Graph

A dependency graph might look like:

```
Application
    ↓
Framework
    ↓
Crypto Library
    ↓
OpenSSL
    ↓
Algorithm
```

This allows ECDAT to identify indirect cryptographic dependencies.

---

33. Cryptographic Usage Graph

Example:

```
Application
    ↓
Function
    ↓
Crypto API
    ↓
Library
    ↓
Algorithm
```

This gives technical provenance.

---

34. PKI Trust Graph

Example:

```
Root CA
   ↓
Intermediate CA
   ↓
Issuing CA
   ↓
Certificate
   ↓
Application
```

ECDAT can answer:

> Which applications depend on this CA?

---

35. Certificate Graph

Consider:

```
Certificate
│
├── issued by → CA
├── contains → Public Key
├── uses → ECDSA
├── protects → TLS Service
└── deployed on → Load Balancer
```

Now certificate risk becomes contextual.

---

36. Key Dependency Graph

Example:

```
HSM
 ↓
Key
 ↓
Certificate
```

```
 ↓
Application
 ↓
Business Process
```

If the key becomes unavailable:

```
Business Process
       ↑
Application
       ↑
Certificate
       ↑
Key
       ↑
HSM
```

ECDAT can calculate the blast radius.

---

37. Application-to-Crypto Graph

Example:

```
Payment API
│
├── TLS → ECDSA
├── Database → AES
├── Signing → RSA
└── Token → HMAC
```

Now ECDAT can produce a complete cryptographic profile of the application.

---

38. Data-to-Crypto Graph

Example:

```
Customer Records
      ↓
Database
      ↓
AES-256-GCM
      ↓
Encryption Key
```

```
      ↓
HSM
```

This allows the question:

> What cryptography protects this data?

---

39. Business-to-Crypto Graph

This is where ECDAT becomes strategically powerful.

Example:

```
Online Payments
      ↓
Payment Gateway
      ↓
TLS
      ↓
ECDSA
      ↓
Certificate
      ↓
CA
```

If ECDSA requires migration:

```
Business Process
        ↓
Migration Requirement
```

Now leadership can see why the cryptographic issue matters.

---

40. Risk Propagation

Suppose:

```
RSA
 ↓
Certificate
 ↓
Payment API
```

```
      ↓
Payment Processing
```

If RSA receives:

```
Risk = HIGH
```

ECDAT can propagate the implication.

Not:

```
Payment Processing = HIGH
```

automatically.

Instead:

```
Potential Business Impact:
High

Reason:
Dependency chain from RSA
to payment processing.
```

The system must distinguish direct risk from propagated impact.

---

41. Blast Radius

One of the most useful graph features:

Blast Radius Analysis

Select:

```
RSA-2048
```

ECDAT returns:

```
73 Applications
18 Certificates
4 CAs
2 Business Processes
3 HSM Dependencies
```

This is much more actionable than a flat inventory.

---

42. Centrality

Graph centrality identifies highly connected cryptographic components.

Example:

```
Library X
   ↓
Used by 2,000 applications
```

That library becomes a strategic migration priority.

Potential metric:

```
Crypto Dependency Centrality
```

High centrality means:

> Changing this component may affect a large portion of the enterprise.

---

43. Single Points of Failure

Suppose:

```
100 applications
      ↓
One HSM
```

That HSM becomes a potential:

```
Cryptographic Single Point of Failure
```

ECDAT should flag this.

Similarly:

```
One CA
 ↓
10,000 certificates
```

creates substantial concentration risk.

---

44. Shared Cryptographic Dependencies

Example:

```
Application A ─┐
Application B ─┤
Application C ─┼──→ OpenSSL
Application D ─┤
Application E ─┘
```

If OpenSSL requires migration:

```
Migration Impact:
Very High
```

This helps prioritize enterprise-wide upgrades.

---

45. Attack Path Analysis

ECDAT can eventually identify paths such as:

```
Internet
 ↓
TLS Endpoint
 ↓
Certificate
 ↓
Private Key
 ↓
HSM
 ↓
Critical Application
```

This does not automatically mean the path is exploitable.

It represents:

> **A relationship path relevant to security analysis.**

---

46. Migration Dependency Graph

Migration itself can become a graph.

Example:

```
HSM Upgrade
     ↓
Crypto Library Upgrade
```

```
        ↓
TLS Stack Upgrade
        ↓
Certificate Migration
        ↓
Application Update
        ↓
Client Migration
```

ECDAT can derive the correct sequence.

---

47. Crypto-Agility Graph

The graph can also model agility.

Example:

```
Application
   ↓
Crypto Abstraction Layer
   ↓
Provider Interface
   ↓
Algorithm
```

This architecture is highly agile.

Compare:

```
Application
   ↓
Hardcoded RSA
```

The second has low agility.

---

48. Knowledge Graph Queries

The graph should support questions such as:

```
Which systems use RSA?

Which critical systems use ECC?

Which applications depend on OpenSSL?
```

```
Which certificates use ECDSA?

Which keys are stored in HSM X?

Which business processes depend on a specific CA?

Which assets have no PQC migration path?
```

---

49. Natural Language Queries

This is where AI can create an impressive interface.

A security analyst could ask:

> "Show me every critical application using quantum-vulnerable cryptography."

ECDAT translates that into graph queries.

Conceptually:

```
Natural Language
        ↓
Intent Extraction
        ↓
Graph Query Generation
        ↓
Graph Database
        ↓
Results
        ↓
AI Explanation
```

---

50. Example Query 1

User:

> "Which critical systems still use RSA?"

Graph logic:

```
Application
WHERE Criticality = Critical
↓
USES
```

```
    ↓
    RSA
```

Output:

```
    42 Critical Applications
```

---

51. Example Query 2

User:

> "What breaks if CA-01 is unavailable?"

ECDAT traverses:

```
    CA-01
     ↓
    Certificates
     ↓
    Applications
     ↓
    Business Processes
```

Output:

```
    127 applications
    14 business processes
```

with evidence.

---

52. Example Query 3

User:

> "Which cryptographic libraries are the biggest migration bottlenecks?"

Graph analysis:

```
    Library
    ↓
    Number of dependent applications
    ↓
    Risk
```

```
↓
Migration complexity
```

Output:

```
Library X
Centrality:
Very High

Affected:
2,140 applications
```

---

53. Example Query 4

User:

> "Which sensitive data is protected by quantum-vulnerable encryption?"

Graph traversal:

```
Sensitive Data
 ↓
Protected By
 ↓
Algorithm
 ↓
Quantum Vulnerability
```

This directly addresses the core SIH problem.

---

54. Example Query 5

User:

> "Give me the top five cryptographic migrations we should start this quarter."

ECDAT combines:

```
Risk
+
Business Criticality
+
Migration Complexity
+
```

```
Dependency Centrality
+
Mosca Assessment
```

and returns a ranked list.

---

55. Graph + AI

The most powerful architecture combines:

```
Graph
+
LLM
+
Deterministic Rules
```

The graph supplies factual relationships.

The LLM interprets and explains them.

Rules enforce constraints.

---

56. Graph RAG

Traditional RAG:

```
Question
↓
Vector Search
↓
Documents
↓
LLM
```

Graph RAG:

```
Question
↓
Entity Extraction
↓
Graph Traversal
↓
Relevant Nodes
↓
```

```
Relevant Documents
↓
LLM
```

This is particularly suitable for ECDAT.

---

57. AI Graph Reasoning

Suppose the analyst asks:

> "Why is this certificate high risk?"

ECDAT can retrieve:

```
Certificate
↓
ECDSA
↓
Critical Application
↓
Long-lived Data
↓
High Migration Complexity
```

The LLM can explain:

> The certificate is high priority because it uses a quantum-vulnerable signature mechanism and protects a critical application with a long migration horizon.

The graph provides the evidence.

---

58. Graph-Based Recommendations

Recommendations can use graph context.

Example:

```
Algorithm:
ECDSA

Application:
Critical

Dependencies:
HSM
```

```
CA
TLS

Migration Complexity:
High
```

Recommendation:

```
Hybrid / staged migration
```

The recommendation is now based on actual enterprise topology.

---

59. Graph-Based Risk

Risk can also propagate through graph relationships.

Example:

```
Algorithm
 ↓
Library
 ↓
Application
 ↓
Business Process
```

A risk engine can calculate:

```
Direct Risk
+
Dependency Impact
+
Business Impact
```

This creates a much richer enterprise risk model.

---

60. Temporal Graph

The graph should not only represent:

> What exists now?

It should represent:

> What existed when?

Example:

```
2026:
RSA

2027:
RSA + ML-KEM

2029:
ML-KEM

2030:
RSA removed
```

This enables migration tracking.

---

61. Historical Tracking

ECDAT should preserve:

```
First Seen
Last Seen
Previous Value
Current Value
Changed By
Scan ID
```

Example:

```
ECDSA:
First Seen: 2026-08-01
Last Seen: 2026-08-24
Status: Active
```

---

62. Change Detection

Suppose:

```
Yesterday:
RSA
```

```
Today:
ML-DSA
```

ECDAT should detect:

```
Cryptographic Change Detected
```

Then recalculate:

```
Risk
CBOM
Migration Status
```

---

63. Versioning

Everything important should be versioned:

```
Repository
Application
Certificate
Library
Algorithm Configuration
Risk
Recommendation
Migration Plan
```

This creates an audit trail.

---

64. Graph Data Quality

Graphs can become messy.

Potential problems:

```
Duplicate Applications
Duplicate Libraries
Unknown Owners
Conflicting Algorithms
Missing Relationships
Stale Assets
```

ECDAT therefore needs data-quality controls.

---

65. Deduplication

Suppose scanners detect:

```
OpenSSL 3.0
OpenSSL/3.0
openssl-3.0.x
```

These may refer to the same dependency.

ECDAT needs canonicalization.

---

66. Entity Resolution

Different scanners may identify:

```
Payment-Service
payment_service
payment-service-prod
```

as separate assets.

Entity resolution can infer:

```
Same Application
```

but should retain evidence and confidence.

---

67. Confidence

Graph relationships should have confidence.

Example:

```
Application
   │
   └── USES → RSA
               Confidence: 0.99
```

Another:

```
Application
   │
   └── USES → ECDSA
               Confidence: 0.61
```

This prevents uncertain relationships from appearing equally authoritative.

---

68. Provenance

Every graph relationship should answer:

> Where did this information come from?

Possible sources:

```
Source Code
Binary
Certificate
Container
Configuration
Runtime Telemetry
Scanner
AI
Manual Entry
```

Example:

```
USES → RSA

Source:
Static Analysis

Evidence:
signing.c:284
```

---

69. Graph Database Architecture

A property graph database is a natural fit.

Possible technology:

Neo4j

The graph can model:

```
Nodes
+
Relationships
```

```
      +
      Properties
```

and support traversal queries.

Other graph databases could also be considered.

The architecture should avoid making the entire system dependent on a single database implementation too early.

---

70. Neo4j / Property Graph

Example conceptual model:

```
      (:Application)
            -[:USES]->
      (:Algorithm)
```

Another:

```
      (:Certificate)
            -[:ISSUED_BY]->
      (:CA)
```

And:

```
      (:Application)
            -[:PROTECTS]->
      (😀ataAsset)
```

---

71. Relational Database Integration

Not everything needs to live in the graph.

Relational databases remain useful for:

```
      Users
      Organizations
      Jobs
      Scans
      Configuration
      Audit Records
      Authentication
```

Therefore:

```
PostgreSQL
+
Graph Database
```

may be a strong architecture.

---

72. Vector Database Integration

Vectors can support semantic retrieval.

For example:

```
Code Embeddings
Documentation Embeddings
Security Guidance
PQC Knowledge
```

Then:

```
Graph
+
Vector Search
+
LLM
```

creates a powerful hybrid knowledge system.

---

73. Search Architecture

ECDAT could support three search modes:

Exact

```
RSA-2048
```

Semantic

```
Find systems performing digital signing.
```

Graph

```
What depends on CA-01?
```

All three should converge on the same intelligence layer.

---

74. Graph API

The frontend should not directly manipulate the graph database.

Instead:

```
Frontend
   ↓
Graph API
   ↓
Query Service
   ↓
Graph Database
```

The API can enforce:

- Authentication
- Authorization
- Query limits
- Data filtering
- Audit logging

---

75. Graph Visualization

A visual graph can show:

```
Application
      |
      ▼
Certificate
      |
      ▼
CA
      |
      ▼
HSM
```

Users should be able to:

- Expand nodes
- Filter by type

- Filter by risk
- Highlight quantum-vulnerable assets
- Trace dependencies
- View evidence

---

76. Enterprise Dashboard

A possible dashboard:

```
┌──────────────────────────────────────────────┐
│        CRYPTOGRAPHIC ENTERPRISE MAP          │
├──────────────────────────────────────────────┤
│                                              │
│ Applications        4,820                    │
│ Crypto Assets       28,391                   │
│ Certificates        74,201                   │
│ Critical Risks      218                      │
│ PQC Ready           31%                      │
│                                              │
└──────────────────────────────────────────────┘
```

Then a graph view:

```
        ┌──────────┐
        │  CA-01   │
        └──────────┘
             │
        ┌────┴─────┐
        ▼          ▼
    Cert-A      Cert-B
      │           │
      ▼           ▼
  Payment       API
      │           │
      └─────┬─────┘
            ▼
        RSA/ECDSA
```

---

77. Security

The graph itself can contain highly sensitive information.

It may reveal:

```
Infrastructure
Keys
Certificates
Dependencies
Business Processes
Security Architecture
```

Therefore the graph must be treated as sensitive security data.

---

78. Access Control

Different users should see different information.

Example:

Developer

```
Own Applications
Own Findings
```

Security Analyst

```
Enterprise Crypto Findings
```

CISO

```
Enterprise Risk
```

Executive

```
Aggregated Business Impact
```

---

79. Sensitive Relationships

Some relationships may be particularly sensitive.

Example:

```
HSM
 ↓
Root CA
```

This should not necessarily be visible to every user.

ECDAT should support relationship-level access control where required.

---

80. Graph Threat Model

The knowledge graph itself can become a target.

Threats include:

- Unauthorized access
- Graph manipulation
- False relationships
- Data poisoning
- Enumeration
- Credential theft
- Sensitive topology disclosure

Therefore graph integrity is critical.

---

81. Scalability

Large organizations may have:

```
Millions
of files

Hundreds of thousands
of certificates

Thousands
of applications

Millions
of relationships
```

The architecture must support incremental ingestion.

Avoid rebuilding the entire graph after every scan.

---

82. Graph Processing Pipeline

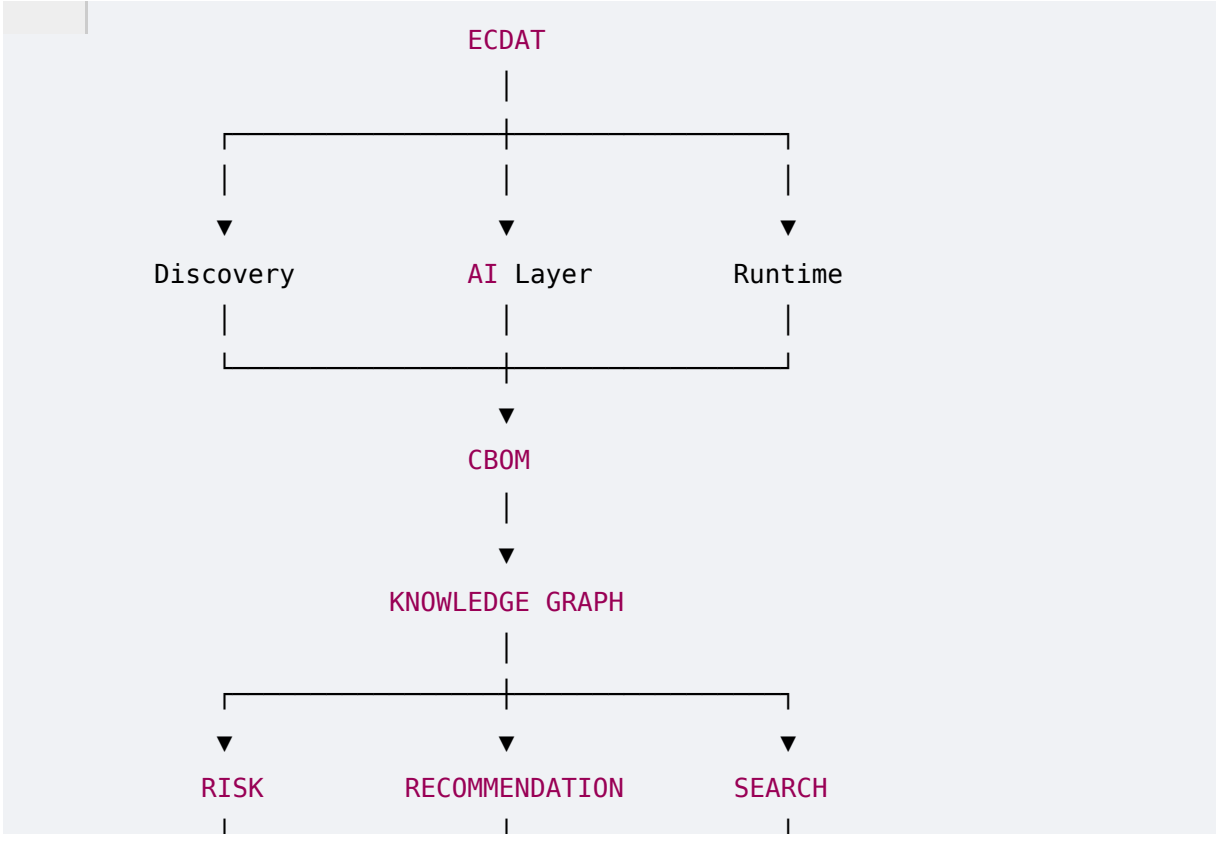
A scalable ingestion model:

```
Scanner
   ↓
Event / Message Queue
   ↓
Normalization
   ↓
Entity Resolution
   ↓
Relationship Validation
   ↓
Graph Upsert
   ↓
Risk Recalculation
   ↓
Index Update
```

---

83. Complete Architecture

The complete intelligence architecture now looks like:



---

84. Requirements Derived From Phase 9

ECDAT should now include:

Graph

- Property graph
- Node model
- Relationship model
- Evidence
- Provenance
- Confidence
- Temporal history

Intelligence

- Dependency analysis
- Blast radius
- Centrality
- Risk propagation
- PKI mapping
- Business mapping

Search

- Exact search
- Semantic search
- Graph search
- Natural-language queries

AI

- Graph RAG
- Natural-language-to-query
- Graph explanation
- Evidence-grounded reasoning

Visualization

- Enterprise graph
- Dependency explorer
- Risk overlay
- Migration graph

Governance

- Access control
- Audit
- Data quality
- Entity resolution
- Versioning

---

85. Phase 9 Summary

Phase 9 fundamentally changes the nature of ECDAT.

Without a graph:

```
RSA-2048
Risk: High
```

With a graph:

```
RSA-2048
│
├── Used by → 73 applications
│
├── Protects → 12 sensitive datasets
│
├── Appears in → 18 certificates
│
├── Depends on → OpenSSL
│
├── Stored through → HSM-04
│
├── Impacts → 4 business processes
│
└── Migration → Blocked by HSM compatibility
```

The second is an actual **enterprise intelligence model**.

---

86. The Most Important Insight

ECDAT should not think in terms of:

> "Cryptographic assets."

It should think in terms of:

Cryptographic relationships.

Because the actual security question is rarely:

> "Does RSA exist?"

It is:

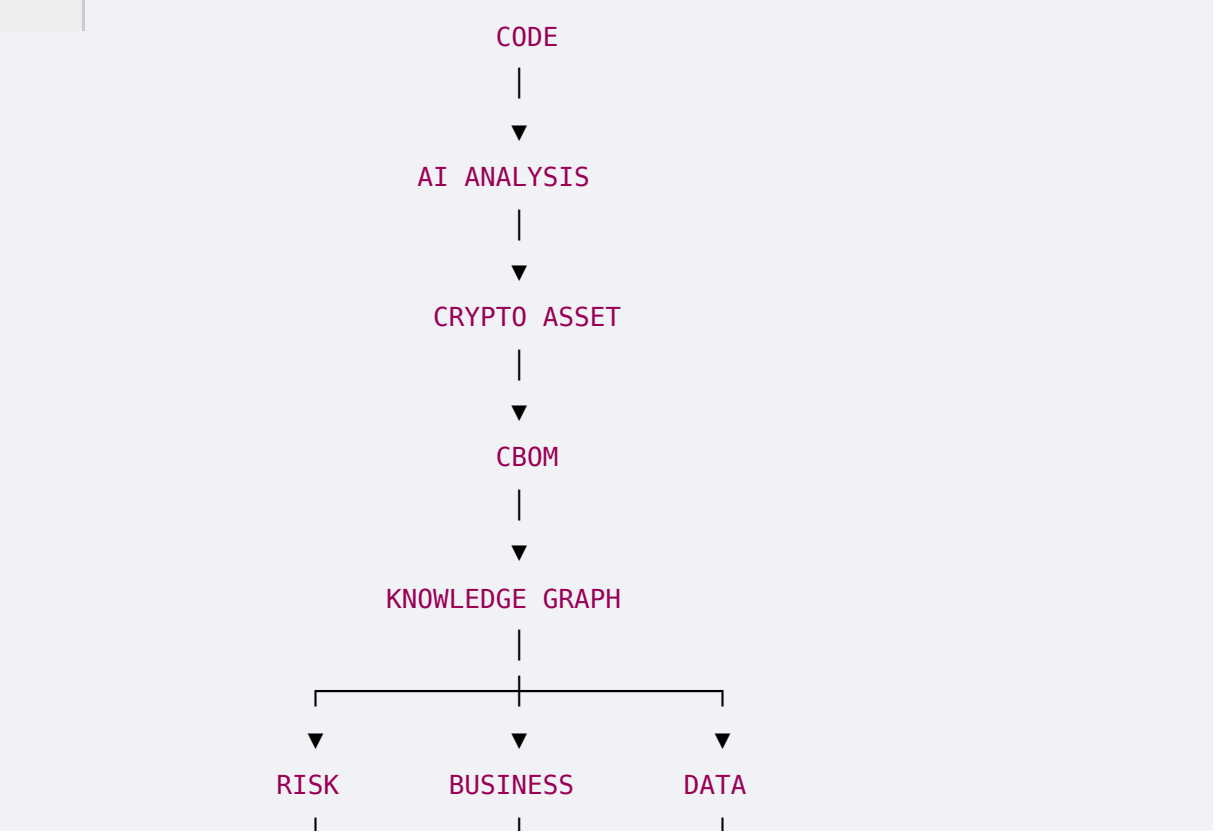
> **"Where is RSA used, what does it protect, who depends on it, how critical is it, and what happens if we need to replace it?"**

That requires a graph.

---

87. The ECDAT Intelligence Chain

After Phase 9, the architecture can now be described as:



This is becoming a full **cryptographic intelligence platform**, rather than simply a scanner.

PHASE 10 PREVIEW — ENTERPRISE SCANNING ENGINE & MULTI-SOURCE DISCOVERY FABRIC

The graph tells us **how everything is connected**.

But now we need to build the machinery that continuously feeds it.

Phase 10 will cover the actual enterprise scanning architecture:

- GitHub/GitLab repository scanning
- Source-code scanning
- Binary scanning
- Container scanning
- Package scanning
- Dependency scanning
- Certificate discovery
- Key metadata discovery
- TLS endpoint scanning
- Network discovery
- Cloud discovery
- Kubernetes discovery
- HSM/KMS discovery
- Configuration scanning
- SBOM integration
- Runtime telemetry
- Endpoint agents
- CI/CD integration
- Scheduled scanning
- Incremental scanning
- Event-driven scanning
- Scanner orchestration
- Job queues
- Distributed workers
- Scan prioritization
- Air-gapped environments
- Offline scanning
- Credential management
- Secure connectors
- Evidence normalization
- Deduplication

- Scan coverage metrics
- Scanner health
- Failure recovery
- Enterprise-scale architecture

The central question will be:

> **How does ECDAT actually discover cryptography across an entire enterprise, continuously and reliably?**

That is where we move from **intelligence architecture** into the **actual scanning infrastructure**.

Phase 9 complete.